Inventory Management System

System Architecture & Technical Overview

Baubek Serikbay, Mardan Khalilov, Daniyaruly Abubakir

System Architecture Overview

Module / Folder	Role	Key Responsibilities & Components
Root Files	Entry Point & Docs	Runs the CLI and role-based routing (main.py); provides central project documentation (README.md).
models/	Data Structures	Blueprints for inventory items (product.py) and system users (user.py).
services/	Business Logic	Core engines for managing products (inventory_mgr.py) and authenticating users (user_mgr.py).
utils/	Helper Tools	Handles File I/O (file_handler.py), system auditing (logger.py), and security (decorators.py).
data/	Database Storage	Persistent JSON storage for products and users, alongside reporting outputs (CSV) and audit logs (TXT).
tests/	Quality Assurance	Automated checks ensuring system reliability, data integrity, and safe stock updates (test_inventory.py).

User Access & Security

Identity Management

user.py defines the base User class alongside specific Admin and Customer subclasses, dictating menu visibility across the application.
user_mgr.py acts as the authentication engine, verifying passwords and safely loading profiles from the JSON database.

Security Checkpoints

main.py handles the initial login routing and intelligently restricts CLI options based on the authenticated user's role.
decorators.py enforces security via the *@admin_required* wrapper, ensuring only authorized administrators can trigger sensitive system functions.

Inventory Management Engine

Product Blueprints

Defined in **product.py**, this component acts as the foundational structure for items (ID, name, price, quantity). It provides self-contained logic for safe stock-level tracking and updates.

Business Logic

Operating via **inventory_mgr.py**, this engine handles the heavy lifting. It executes core requirements: adding, removing, updating, searching, and filtering products efficiently.

Data Persistence Strategy

- **File I/O Management:** *file_handler.py* provides reusable functions to safely load and save system state according to the required JSON input format.
- **Core State Storage:** *inventory.json* and *users.json* act as the persistent database, ensuring product and credential data is preserved between sessions.
- **Reporting & Outputs:** The system generates structured reports, such as low stock alerts or full *inventory_report.csv* exports, meeting core output requirements.

Quality Assurance & Auditing

System Auditing

logger.py serves as the system's objective observer. It records critical actions—such as adding products or updating stock—with precise timestamps.
These immutable records are continuously appended to **audit_log.txt**, creating a transparent, running history of operations.

Automated Testing

test_inventory.py functions as the dedicated quality assurance suite, running rigorous automated checks against the inventory manager.
It analytically proves that core features function correctly and ensures the system handles invalid data inputs gracefully without crashing.